Report 2: Routy - A routing network

Rui Zhang

September 16, 2025

1 Introduction

This report covers the implementation of a routing network that can use link state protocols to route messages between nodes in the network. To create the routing table, Dijkstra's algorithm was used. The main topic discussed in this work is to gain a deeper understanding of how to communicate in a network containing multiple nodes/servers, which are distributed over a vast area and do not have direct access between nodes. This requires the system to be able to track network information and make frequent changes to the system. The system should have fault tolerance and be able to handle situations where certain nodes go offline without affecting the system's performance.

2 Main problems and solutions

The system consists of the following multiple files:

map.erl

Maps are used to help identify which nodes are directly connected to each other. However, due to the possibility of nodes appearing and disappearing, we need to update the map accordingly.

dijkstra.erl

The real challenge of this job is to make the routing table work properly. To obtain the gateway for routing messages to any node, a sorted list was used to track all entries. In order to update the shortest path, several functions were used, mainly including entry, update, iteration, table, replace, route and so on.

intf.erl

All references and process identifiers of different routers are tracked through interfaces. The interface is described by symbol name (city), process reference, and process identifier. This file also contains numerous functions for obtaining names, references, or process identifiers, which are necessary in different situations of the system.

hist.erl

One issue faced when implementing link state protocols is to resend messages that have already been sent. This problem is solved through hist. erl, which processes messages that have already been sent as old messages. If the message is new, it updates the history list to track the messages sent to each node.

routy.erl

This module is a simple link state router that simulates the behavior of a router using Erlang processes and performs communication and routing calculations through message passing. This module fully implements a link state router process model, supporting functions such as connection, state broadcasting, routing calculation, message forwarding, monitoring downtime, and status query. It also collaborates with internal modules such as intf.erl, map.erl, hist.erl, dijkstra.erl to complete real routing behavior.

routy_test.erl

This Erlang module code routy_test is a complete automated testing script used to demonstrate the functionality of the routing protocol I previously built. It implements functions such as automated creation, link state propagation, shortest path routing, message routing testing, fault tolerance, state output, and cleaning.

distributed_test.erl

This Erlang module distributed_test implements a toolkit for testing and managing distributed routing networks, particularly suitable for the routing protocol I previously wrote. It simulates intercontinental networks (Europe, Asia, America, etc.) to initiate, connect, broadcast, update, send messages, simulate node failures, and perform other operations. It implements the functions of starting the router, connecting the area, broadcasting and updating, sending messages, fault simulation, network viewing and cleaning.

3 Evaluation

map.erl

```
Erlang/OTP 28 [erts-16.0.2] [source] [64-bit] [smp:32:32] [ds:32:32:10] [async-threads:1] [jit:ns]

Eshell V16.0.2 (press Ctrl+G to abort, type help(). for help)
1> cd("D:/java tool/IDEA project/erlang-2/src").
d:/java tool/IDEA project/erlang-2/src
ok
2>  map:new().
[]
3> map:update(berlin, [london, paris], []).
[{berlin,[london,paris]}]
4>  map:reachable(berlin, [{berlin,[london,paris]}]).
[london,paris]
5> map:reachable(london, [{berlin,[london,paris]}]).
[]
6> map:all_nodes([{berlin,[london,paris]}]).
[berlin,london,paris]
7> map:update(berlin, [madrid], [{berlin,[london,paris]}]).
[{berlin,[madrid]}]
8>
```

dijkstra.erl

```
8> dijkstra:update(london, 2, amsterdam, []).
[]
9>  dijkstra:update(london, 2, amsterdam, [{london, 2, paris}]).
[{london,2,paris}]
10> dijkstra:update(london, 1, stockholm,[{berlin, 2, paris}, {london, 3, paris}]).
[{london,1,stockholm},{berlin,2,paris}]
11> dijkstra:iterate([{paris, 0, paris}, {berlin, inf, unknown}], [{paris, [berlin]}], []).
[{berlin,paris},{paris,paris}]
12> dijkstra:table([paris, madrid], [{madrid,[berlin]}, {paris, [rome,madrid]}]).
[{berlin,madrid},{rome,paris},{paris,paris},{madrid,madrid}]
```

routy_test.erl

I have finished an encapsulated test file routy_test.erl for the code routy.erl that I have written, which sets up a network containing at least five routers and demonstrates routy's functionality, including message broadcasting, link-state updating, computing routing table and rerouting after node failure.

The 5 routers are stockholm, goteborg, malmo, lund and uppsala.

```
1> cd("D:/java tool/IDEA project/erlang-2/src").
d:/java tool/IDEA project/erlang-2/src
ok
2> routy_test:test_network().
=== Starting Routy Network Test ===
请确保已编译所有模块: map, intf, hist, dijkstra, routy

1. Creating routers...

2. Establishing network connections...
stockholm: adding interface to goteborg
goteborg: adding interface to stockholm
malmo: adding interface to goteborg
lund: adding interface to malmo
uppsala: adding interface to stockholm
stockholm: adding interface to uppsala
goteborg: adding interface to malmo
malmo: adding interface to lund
lund: adding interface to stockholm

3. Broadcasting link-state messages...
stockholm: broadcasting link-state message
uppsala: received link-state message from stockholm
goteborg: received link-state message from stockholm
uppsala: new link-state message, broadcasting
goteborg: new link-state message, broadcasting
stockholm: received link-state message from stockholm
malmo: received link-state message from stockholm
stockholm: old link-state message, ignoring
malmo: new link-state message, broadcasting
stockholm: received link-state message from stockholm
lund: received link-state message from stockholm
goteborg: received link-state message from stockholm
stockholm: old link-state message, ignoring
lund: new link-state message, broadcasting
goteborg: old link-state message, ignoring
malmo: received link-state message from stockholm
stockholm: received link-state message from stockholm
malmo: old link-state message, ignoring
stockholm: old link-state message, ignoring
goteborg: broadcasting link-state message
malmo: received link-state message from goteborg
stockholm: received link-state message from goteborg
malmo: new link-state message, broadcasting
stockholm: new link-state message, broadcasting
lund: received link-state message from goteborg
goteborg: received link-state message from goteborg
uppsala: received link-state message from goteborg
lund: new link-state message, broadcasting
goteborg: old link-state message, ignoring
uppsala: new link-state message, broadcasting
stockholm: received link-state message from goteborg
```

```
4. Updating routing tables...
stockholm: updating routing table
goteborg: updating routing table
malmo: updating routing table
lund: updating routing table
uppsala: updating routing table
stockholm: routing table updated with gateways [uppsala,goteborg]
goteborg: routing table updated with gateways [malmo,stockholm]
malmo: routing table updated with gateways [lund,goteborg]
lund: routing table updated with gateways [stockholm,malmo]
uppsala: routing table updated with gateways [stockholm]

5. Testing message routing...
stockholm: sending message '[72,101,108,108,111,32,102,114,111,109,32,83,116,111,99,107,104,111,108,109,32,116,111,32,76,117,110,100,33]' to lund
stockholm: routing message '[72,101,108,108,111,32,102,114,111,109,32,83,116,111,99,107,104,111,108,109,32,116,111,32,76,117,110,100,33]' from stockholm to lund
stockholm: forwarding via gateway goteborg
goteborg: routing message '[72,101,108,108,111,32,102,114,111,109,32,83,116,111,99,107,104,111,108,109,32,116,111,32,76,117,110,100,33]' from stockholm to lund
goteborg: forwarding via gateway malmo
malmo: routing message '[72,101,108,108,111,32,102,114,111,109,32,83,116,111,99,107,104,111,108,109,32,116,111,32,76,117,110,100,33]' from stockholm to lund
malmo: forwarding via gateway lund
lund: received message '[72,101,108,108,111,32,102,114,111,109,32,83,116,111,99,107,104,111,108,109,32,116,111,32,76,117,110,100,33]' from stockholm
stockholm: sending message '[68,105,114,101,99,116,32,109,101,115,115,97,103,101,32,116,111,32,85,112,112,115,97,108,97]' to uppsala
stockholm: routing message '[68,105,114,101,99,116,32,109,101,115,115,97,103,101,32,116,111,32,85,112,112,115,97,108,97]' from stockholm to uppsala
stockholm: forwarding via gateway uppsala
uppsala: received message '[68,105,114,101,99,116,32,109,101,115,115,97,103,101,32,116,111,32,85,112,112,115,97,108,97]' from stockholm
malmo: sending message '[82,101,112,108,121,32,102,114,111,109,32,77,97,108,109,111,32,116,111,32,83,116,111,99,107,104,111,108,109]' to stockholm
malmo: routing message '[82,101,112,108,121,32,102,114,111,109,32,77,97,108,109,111,32,116,111,32,83,116,111,99,107,104,111,108,109]' from malmo to stockholm
```

```
6. Final network status:
stockholm: status requested
Router stockholm (counter: 1)
  Interfaces: [uppsala,goteborg]
  Routing table: [{lund,goteborg},{malmo,goteborg},{stockholm,goteborg},{uppsala,uppsala},{goteborg,goteborg}]

goteborg: status requested
Router goteborg (counter: 1)
  Interfaces: [malmo,stockholm]
  Routing table: [{lund,malmo},{goteborg,malmo},{uppsala,stockholm},{stockholm,stockholm},{malmo,malmo}]

malmo: status requested
Router malmo (counter: 1)
  Interfaces: [lund,goteborg]
  Routing table: [{uppsala,goteborg},{malmo,goteborg},{stockholm,goteborg},{lund,lund},{goteborg,goteborg}]

lund: status requested
Router lund (counter: 1)
  Interfaces: [stockholm,malmo]
  Routing table: [{lund,malmo},{goteborg,malmo},{uppsala,stockholm},{stockholm,stockholm},{malmo,malmo}]

uppsala: status requested
Router uppsala (counter: 1)
  Interfaces: [stockholm]
  Routing table: [{lund,stockholm},{malmo,stockholm},{uppsala,stockholm},{goteborg,stockholm},{stockholm,stockholm}]


7. Testing node failure (stopping goteborg)...
goteborg: stopping router
malmo: exit received from goteborg
stockholm: exit received from goteborg

8. Updating network after failure...
stockholm: broadcasting link-state message
malmo: broadcasting link-state message
lund: broadcasting link-state message
uppsala: broadcasting link-state message
lund: received link-state message from malmo
stockholm: received link-state message from lund
malmo: received link-state message from lund
uppsala: received link-state message from stockholm
lund: new link-state message, broadcasting
stockholm: new link-state message, broadcasting
malmo: new link-state message, broadcasting
uppsala: new link-state message, broadcasting
stockholm: received link-state message from uppsala
malmo: received link-state message from malmo
lund: received link-state message from lund
uppsala: received link-state message from lund
```

```
stockholm: updating routing table
malmo: updating routing table
lund: updating routing table
uppsala: updating routing table
stockholm: routing table updated with gateways [uppsala]
malmo: routing table updated with gateways [lund]
lund: routing table updated with gateways [stockholm,malmo]
uppsala: routing table updated with gateways [stockholm]

9. Testing routing after failure...
stockholm: sending message '[77,101,115,115,97,103,101,32,97,102,116,101,114,32,103,111,116,101,98,111,114,103,32,102,97,105,108,117,114,101]' to lund
stockholm: routing message '[77,101,115,115,97,103,101,32,97,102,116,101,114,32,103,111,116,101,98,111,114,103,32,102,97,105,108,117,114,101]' from stockholm to lund
stockholm: no route to lund found

=== Test completed ===
ok
```

Bonus task

distributed_test.erl

Firstly, I will conduct all tests on a single terminal. And it takes about 1002 ms between a disabled network card and the delivery of a 'DOWN' message to the other nodes.

Europe has 5 routers: stockholm,oslo,berlin,paris,london

Asia has 5 routers: tokyo,beijing,mumbai,bangkok,seoul

```
Eshell V16.0.2 (press Ctrl+G to abort, type help(). for help)
1> cd("D:/java tool/IDEA project/erlang-2/src").
d:/java tool/IDEA project/erlang-2/src
ok
2> distributed_test:setup_region(europe).
=== Setting up europe region ===
Creating routers for europe: [stockholm,oslo,berlin,paris,london]
Connected stockholm <-> oslo
stockholm: adding interface to oslo
oslo: adding interface to stockholm
Connected oslo <-> berlin
oslo: adding interface to berlin
```

```
Region europe setup complete with 5 routers
[stockholm,oslo,berlin,paris,london]
3> distributed_test:setup_region(asia).
=== Setting up asia region ===
Creating routers for asia: [tokyo,beijing,mumbai,bangkok,seoul]
Connected tokyo <-> beijing
tokyo: adding interface to beijing
beijing: adding interface to tokyo
```

```
Region asia setup complete with 5 routers
[tokyo,beijing,mumbai,bangkok,seoul]
4> distributed_test:connect_to_region(london, tokyo).
Connecting london to tokyo
london: adding interface to tokyo
tokyo: adding interface to london
london: broadcasting link-state message
```

```
Connection established: london <-> tokyo
ok
5> distributed_test:broadcast_region(europe).
Broadcasting in europe region...
stockholm: broadcasting link-state message
london: received link-state message from stockholm
```

```
ok
6> distributed_test:broadcast_region(asia).
Broadcasting in asia region...
tokyo: broadcasting link-state message
london: received link-state message from tokyo
```

```
7> distributed_test:update_region(europe).
Updating routing tables in europe region...
stockholm: updating routing table
stockholm: routing table updated with gateways [london,oslo]
oslo: updating routing table
oslo: routing table updated with gateways [berlin,stockholm]
berlin: updating routing table
berlin: routing table updated with gateways [paris,oslo]
paris: updating routing table
paris: routing table updated with gateways [london,berlin]
london: updating routing table
london: routing table updated with gateways [tokyo,stockholm,paris]
ok
8> distributed_test:update_region(asia).
Updating routing tables in asia region...
tokyo: updating routing table
tokyo: routing table updated with gateways [london,seoul,beijing]
beijing: updating routing table
beijing: routing table updated with gateways [mumbai,tokyo]
mumbai: updating routing table
mumbai: routing table updated with gateways [bangkok,beijing]
bangkok: updating routing table
bangkok: routing table updated with gateways [seoul,mumbai]
seoul: updating routing table
seoul: routing table updated with gateways [tokyo,bangkok]
ok
```

```
9> distributed_test:send_message(stockholm, tokyo, "Hello Asia from Europe!").
Sending message from stockholm to tokyo: Hello Asia from Europe!
stockholm: sending message '[72,101,108,108,111,32,65,115,105,97,32,102,114,111,109,32,69,117,114,111,112,101,33]' to tokyo
ok
stockholm: routing message '[72,101,108,108,111,32,65,115,105,97,32,102,114,111,109,32,69,117,114,111,112,101,33]' from stockholm to tokyo
stockholm: forwarding via gateway london
london: routing message '[72,101,108,108,111,32,65,115,105,97,32,102,114,111,109,32,69,117,114,111,112,101,33]' from stockholm to tokyo
london: forwarding via gateway tokyo
tokyo: received message '[72,101,108,108,111,32,65,115,105,97,32,102,114,111,109,32,69,117,114,111,112,101,33]' from stockholm
10> distributed_test:kill_router(london).
Killing router london (PID: <0.91.0>)
london: stopping router
paris: exit received from london
tokyo: exit received from london
stockholm: exit received from london
Router london stopped. Time elapsed: 1002 ms
ok
11> distributed_test:send_message(stockholm, tokyo, "After failure - still works?").
Sending message from stockholm to tokyo: After failure - still works?
stockholm: sending message '[65,102,116,101,114,32,102,97,105,108,117,114,101,32,45,32,115,116,105,108,108,32,119,111,114,107,115,63]' to tokyo
ok
stockholm: routing message '[65,102,116,101,114,32,102,97,105,108,117,114,101,32,45,32,115,116,105,108,108,32,119,111,114,107,115,63]' from stockholm to tokyo
stockholm: gateway london not found in interfaces
```

The above results indicate that the task requirements have been met.

4 Conclusions

This assignment is more difficult than the previous one. It contains more files and code that need to be tracked. In addition, this assignment seems to require more functional programming concepts than previous assignments. The main learning topic is implementing link state routing protocols and using the Dijkstra algorithm. In addition, implementing some necessary functions in the messaging system, such as history recording and confirmation, is also a great learning experience. More challenging features can be implemented in this router, where the recipient's delivery receipt message will be sent as an acknowledgement. Using built-in libraries for list processing is also a new experience for

me, as we wrote these functions ourselves in previous courses.